

Introduction

Organizor is the backend API for managing tenant environments. It keeps tenant records in a central catalog database and provisions a separate PostgreSQL database for each tenant.

The current implementation focuses on tenant lifecycle operations:

- creating tenant catalog records;
- onboarding tenants by creating a tenant database and applying migrations;
- resolving tenant context for tenant-scoped requests;
- upgrading tenant databases by running migrations;
- soft-deleting tenants before hard deletion;
- exposing health checks, OpenAPI JSON, and Scalar API reference in development.

Main Concepts

Concept	Description
Tenant	A customer or isolated workspace tracked by <code>Id</code> , <code>Name</code> , <code>Slug</code> , <code>DatabaseName</code> , lifecycle status, and deletion metadata.
Catalog database	Central PostgreSQL database that stores all tenant records.
Tenant database	PostgreSQL database named from the tenant slug, currently using the <code>tenant_{slug}</code> convention.
Tenant resolution	Middleware step that reads <code>X-Tenant-Slug</code> , loads the tenant, and stores the current tenant in <code>ITenantContext</code> .
Provisioning workflow	Infrastructure workflow that creates the tenant database and applies tenant migrations.

Project Layout

Project	Responsibility
<code>Code7Dev.Organizor.Api</code>	ASP.NET Core host, controllers, DTOs, mappers, middleware, OpenAPI/Scalar, health checks, CORS, and logging setup.
<code>Code7Dev.Organizor.Application</code>	Tenant service contracts, tenant models, onboarding, lifecycle, upgrade, and catalog-facing application

Project	Responsibility
	logic.
<code>Code7Dev.Organizor.Domain</code>	Tenant entity and <code>TenantStatus</code> enum.
<code>Code7Dev.Organizor.Infrastructure</code>	EF Core contexts, repository implementation, PostgreSQL connection string building, database creation/deletion, migration, and tenant context storage.

Generated Reference

DocFX builds API reference pages from the C# projects into the `api` section. Use those pages for class, interface, and method signatures, and use these articles for workflow-level documentation.

Getting Started

Prerequisites

- .NET SDK 10.
- PostgreSQL available to the API.
- A catalog database connection string named `Catalog`.
- PostgreSQL server settings under the `Postgres` configuration section for tenant database creation and resolution.
- DocFX installed as a .NET tool when building documentation locally.

Run the API

From the repository root:

```
dotnet run --project .\Code7Dev.Organizor.Api\Code7Dev.Organizor.Api.csproj
```

On startup the API:

1. configures Serilog;
2. registers catalog and tenant database services;
3. registers application and infrastructure services;
4. seeds or migrates the catalog database;
5. maps controllers, health checks, OpenAPI, and Scalar in development.

Development Endpoints

Endpoint	Purpose
<code>/health</code>	ASP.NET Core health checks.
<code>/openapi/v1.json</code>	OpenAPI document, available in development.
<code>/scalar</code>	Scalar API reference, available in development.

Build the Documentation

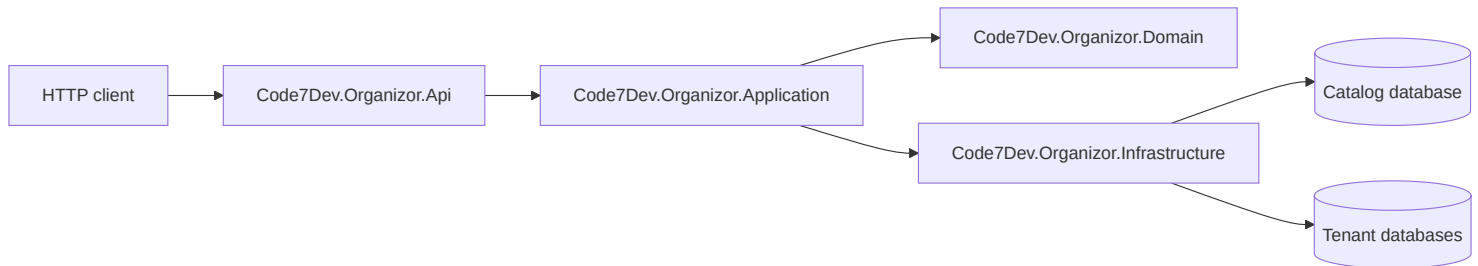
From the repository root:

```
docfx .\docs\docfx.json
```

DocFX writes the generated site to `docs/_site` and generated API metadata to `docs/api`. Both folders are ignored by git and can be rebuilt.

Architecture

Organizor follows a layered structure around tenant management.



API Layer

The API project owns transport concerns:

- controller routes for tenant catalog and tenant operations;
- request and response DTOs;
- mapping between API DTOs and application models;
- correlation ID, exception handling, and tenant resolution middleware;
- Serilog request logging;
- CORS, health checks, OpenAPI, and Scalar setup.

The startup flow in `Program.cs` composes the application by registering catalog persistence, tenant persistence, application services, infrastructure services, and API services.

Application Layer

The application project owns use cases and contracts. It defines interfaces for repository access, tenant database management, migration, provisioning, onboarding, upgrades, and lifecycle operations.

Current services:

Service	Responsibility
<code>TenantService</code>	Creates catalog tenant records and reads tenants by slug or as a list.
<code>TenantOnboardingService</code>	Creates a catalog tenant, runs provisioning, and marks the tenant active or failed.
<code>TenantLifecycleService</code>	Soft-deletes tenants and hard-deletes already soft-deleted tenants.

Service	Responsibility
<code>TenantUpgradeService</code>	Runs migrations for an existing tenant database.

Domain Layer

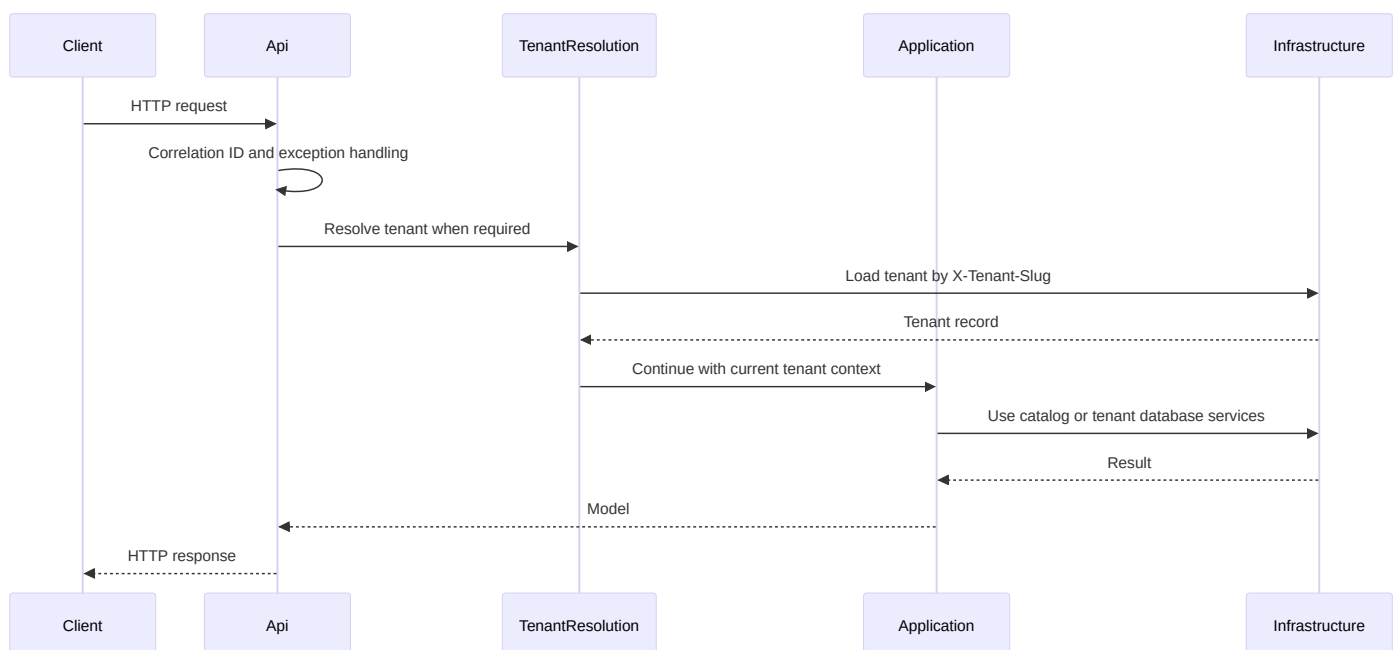
The domain project currently contains the tenant entity and status enum. A tenant stores identity, slug, database name, creation date, lifecycle status, provisioning failure reason, and deletion metadata.

Infrastructure Layer

The infrastructure project owns external resources:

- `CatalogDbContext` stores tenant records.
- `OrganizorDbContext` connects to the current tenant database after tenant resolution.
- `TenantRepository` implements catalog tenant persistence.
- `ConnectionStringBuilder` builds PostgreSQL connection strings from configuration.
- `TenantDatabaseManager` creates and drops tenant databases.
- `TenantMigrator` applies EF Core migrations to tenant databases.
- `TenantProvisioningWorkflow` creates a tenant database and applies migrations during onboarding.

Request Pipeline



Tenancy

Organizor uses catalog-driven, database-per-tenant tenancy.

Tenant Catalog

The catalog database stores tenant metadata through `CatalogDbContext`. Each tenant includes:

- `Id`;
- `Name`;
- `Slug`;
- `DatabaseName`;
- `CreatedAt`;
- `Status`;
- `FailureReason`;
- soft-delete metadata.

The database name convention is `tenant_{slug}`. Slugs are normalized to lowercase before they are stored or used for lookups.

Tenant Statuses

Status	Meaning
Pending	Tenant exists in the catalog but provisioning is not complete.
Active	Tenant provisioning completed successfully.
Upgrading	Reserved for upgrade lifecycle state.
Failed	Provisioning failed or an unexpected onboarding error occurred.
Deleted	Tenant has been soft-deleted.

Tenant Resolution

`TenantResolutionMiddleware` resolves tenant context for tenant-scoped routes.

It skips platform endpoints such as:

- `/api/admin`;
- `/api/v1/tenants`;
- `/scalar`;

- `/openapi`;
- `/health` through route behavior;
- root and common static paths.

For tenant-scoped requests, clients must send:

`X-Tenant-Slug`: acme

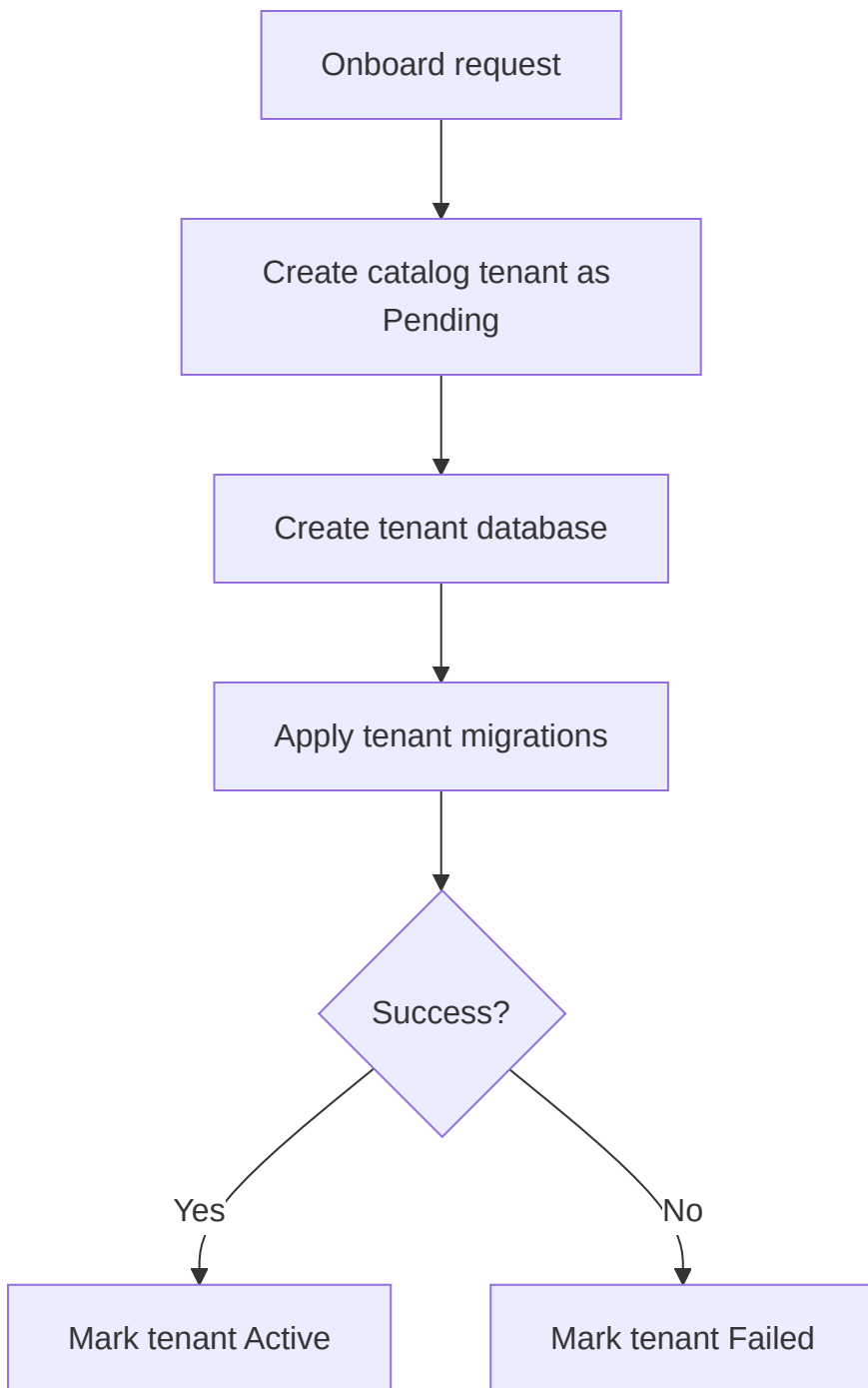
The middleware loads the tenant from the catalog and stores `TenantInfo` in `ITenantContext`. If the header is missing, the response is `400 Bad Request`. If the slug does not match a catalog tenant, the response is `404 Not Found`.

Tenant Database Resolution

`OrganizerDbContext` uses a fallback in-memory database when no tenant has been resolved. This supports startup, Scalar, migrations, and design-time tooling.

When a tenant is resolved, `PerTenantDbResolver` loads the catalog tenant, builds a PostgreSQL connection string for that tenant database, and `OrganizerDbContext` uses that database.

Provisioning Flow



Hard deletion is intentionally guarded: the tenant must be soft-deleted before `TenantLifecycleService` drops the tenant database and removes the catalog record.

API

The API exposes platform-level tenant management endpoints. Tenant-scoped endpoints should include the `X-Tenant-Slug` header unless the path is explicitly skipped by tenant resolution.

Tenant Catalog

Create Tenant

```
POST /api/v1/tenants
Content-Type: application/json
```

```
{
  "name": "Acme",
  "slug": "acme",
  "culture": "en-US"
}
```

Creates a catalog tenant record with `Pending` status and returns tenant metadata. This route does not provision a tenant database.

Get Tenant By Slug

```
GET /api/v1/tenants/acme
```

Returns a tenant by slug, or `404 Not Found` when no tenant exists.

List Tenants

```
GET /api/v1/tenants
```

Returns all catalog tenants visible through the tenant repository.

Tenant Operations

Onboard Tenant

```
POST /api/admin/tenants/onboard
Content-Type: application/json
```

```
{
  "name": "Acme",
```

```
"slug": "acme"
}
```

Creates the catalog tenant, creates the tenant database, applies tenant migrations, and marks the tenant **Active** or **Failed**.

Upgrade Tenant

POST /api/admin/tenants/{id}/upgrade

Runs tenant database migrations for the tenant identified by **id**. Returns a migration result with the database name, success flag, applied migration count, duration, and optional error.

Soft Delete Tenant

```
DELETE /api/admin/tenants/{id}
Content-Type: application/json

{
  "reason": "Customer cancellation"
}
```

Marks the tenant as deleted in the catalog. The tenant database is not dropped during soft deletion.

Hard Delete Tenant

POST /api/admin/tenants/{id}/hard-delete

Drops the tenant database and removes the catalog record. The tenant must already be soft-deleted.

Local API Reference

When the API runs in development:

- OpenAPI JSON is available at **/openapi/v1.json**.
- Scalar is available at **/scalar**.

The DocFX API navigation tab contains the generated C# reference for controllers, DTOs, services, interfaces, middleware, and persistence types.

Configuration

Organizor reads standard ASP.NET Core configuration from `appsettings.json`, environment-specific files, user secrets, and environment variables.

Catalog Connection String

`CatalogDbContext` expects a connection string named `Catalog`.

Example shape:

```
{
  "ConnectionStrings": {
    "Catalog":
    "Host=localhost;Port=5432;Database=organizor_catalog;Username=postgres;Password=postgres"
  }
}
```

Tenant PostgreSQL Settings

`ConnectionStringBuilder` builds tenant and admin connection strings from the `Postgres` section.

```
{
  "Postgres": {
    "Host": "localhost",
    "Port": "5432",
    "User": "postgres",
    "Password": "postgres"
  }
}
```

The admin connection uses the `postgres` database. Tenant connections use the tenant database name stored in the catalog.

Logging

Serilog is configured in `appsettings.json`.

Current sinks:

- console;
- rolling file logs under `logs/log-.txt`.

Log output includes tenant and request correlation properties when available:

- `Id`;
- `Slug`;
- `DatabaseName`;
- `CorrelationId`.

CORS

The API selects a CORS policy by environment:

- `DevCors` in development;
- `ProdCors` outside development.

See `CorsExtensions` for the concrete allowed origins and policy behavior.

OpenAPI and Scalar

OpenAPI and Scalar endpoints are mapped only in development. This keeps the production surface smaller while retaining local API exploration during development.

Operations

Documentation Deployment

The repository contains a GitHub Actions workflow at `.github/workflows/docfx-deploy.yml`.

On pushes to `master`, it:

1. checks out this repository;
2. installs DocFX;
3. builds `docs/docfx.json`;
4. checks out the public documentation repository;
5. copies `docs/_site` into that repository;
6. commits and pushes the generated documentation.

The workflow uses the `DOCS_DEPLOY_KEY` secret to push to the public docs repository.

Generated Documentation Folders

These paths are generated and ignored by git:

- `docs/_site`;
- `docs/.cache`;
- `docs/api`;

Rebuild them with:

```
docfx .\docs\docfx.json
```

Catalog Database Startup

At application startup, `SeedCatalogDbAsync` checks whether the catalog database exists, creates it when necessary, applies catalog migrations, and logs whether tenant records are present.

Tenant Database Lifecycle

Tenant database operations are performed through infrastructure services:

Operation	Service
Create tenant database	<code>TenantDatabaseManager</code>
Apply tenant migrations	<code>TenantMigrator</code>

Operation	Service
Provision a tenant	TenantProvisioningWorkflow
Drop tenant database	TenantDatabaseManager

Deletion Policy

Hard deletion requires soft deletion first. This protects active tenants from accidental database drops and gives the catalog a clear deletion trail before the physical tenant database is removed.